Datalogger SPQ — Informe de la primera instalación de campo

Firmware FWDLGARM 0.0.19 sobre placa SPQ_ARM_R1 — corrida del 20 al 28 de octubre de 2026

Spymovil — Desarrollo de firmware

30 de octubre de 2026 — rev. 2, con los arreglos aplicados

Índice

1. Resumen ejecutivo **2**
1.1. Lo que se detectó, en una tabla . . . 2
1.2. El plan, en una línea cada uno . . . 2

2. Estado general del firmware **3**
2.1. Dónde está el proyecto . . . 3
2.2. Lo que se ha probado en corridas largas . . . 3
2.3. Lo que falta para producción . . . 3
2.4. Una advertencia sobre el presupuesto de energía . . . 4

3. La corrida: qué se midió y cómo **5**
3.1. Montaje y configuración . . . 5
3.2. El instrumento: el registro de eventos . . . 5
3.3. Ventanas de tiempo . . . 5

4. Lo que quedó validado **6**
4.1. Estabilidad: 188 horas sin un reinicio . . . 6
4.2. Memoria: invariable . . . 6
4.3. Datos: ni un registro perdido . . . 6
4.4. Alimentación: la primera curva real de batería . . . 6
4.5. El riel de 3,3 V: estable . . . 7
4.6. El retiro del equipo validó, de paso, la degradación sin enlace . . . 7

5. Hallazgos **8**
5.1. PRIORIDAD 1 — El reloj del kernel atrasa 2,3 %, y eso sesga el caudal . . . 8
5.2. PRIORIDAD 2 — El cálculo de deriva del reloj desborda y reporta un número inventado . . . 9
5.3. PRIORIDAD 3 — El reloj del módulo LTE entregó una hora errada en 3 horas, y se aplicó . . . 10

6. Los dos casos que quedan cerrados **11**
6.1. El modem que dejó de transmitir: fue el retiro del equipo . . . 11
6.2. La deriva del reloj de tiempo real: resuelta en el PCB nuevo . . . 11

7. Plan de trabajo propuesto **12**
7.1. Prioridades . . . 12
7.2. Notas sobre cada punto . . . 12
7.3. Lo que este informe recomienda NO hacer . . . 13

8. Anexos **14**
8.1. Anexo A — Los descartes del contador de pulsos . . . 14
8.2. Anexo B — Serie de alimentación, por resumen de 6 horas . . . 14

8.3. Anexo C — Uso de memoria, estabilizado 14
8.4. Anexo D — Sesiones con el servidor 15
8.5. Anexo E — Origen de los datos 15

1. Resumen ejecutivo

El equipo estuvo instalado en campo **188 horas (7,8 días) sin un solo reinicio**, transmitiendo solo, y dejó un registro de eventos de 454 líneas en su microSD. Es la primera corrida real del firmware fuera del banco y su propósito era doble: confirmar que el equipo aguanta operación continua desatendida, y usar el registro de eventos para ver *lo que no llega al servidor*.

Veredicto: el firmware pasó. No hubo reinicios, no se perdió ni un registro, la memoria no se movió un byte en nueve días y el 95 % de las sesiones con el servidor cerraron correctamente. El registro de eventos cumplió su función: detectó cosas que de otro modo nadie habría visto.

1.1. Lo que se detectó, en una tabla

	Hallazgo	Estado
1	**El reloj del kernel atrasa 2,3 %** con la carga de pulsos de esta instalación, lo que introduce un **sesgo de +2,3 % en el caudal** que se reporta al servidor	**Parcial**: corregida una parte y **añadido el instrumento** que falta para el resto — ver §7
2	El cálculo de deriva del reloj **desborda y reporta un número inventado**, con el signo invertido	**CORREGIDO** y verificado contra el log real
3	El reloj del módulo LTE entregó una hora **errada en 3 horas** y el firmware la aplicó sin verificarla	**CORREGIDO** y verificado con seis casos
4	El modem dejó de transmitir el 28/10 a las 12:39	**Cerrado**: coincide con el retiro del equipo (antena desconectada)
5	El cristal del reloj de tiempo real deriva +400 ppm (34 s/día)	**Cerrado por hardware**: el PCB nuevo lleva los condensadores de carga que faltaban

1.2. El plan, en una línea cada uno

1. **Corregir el atraso del reloj del kernel** y revalidarlo con un test de 48 h. Es el único hallazgo que afecta a un dato que el cliente ve.
2. **Corregir el desborde del cálculo de deriva** (dos líneas).
3. **Validar el reloj que entrega el módulo LTE** antes de aplicarlo.
4. **Medir la deriva del reloj en el PCB nuevo**, que convierte una alarma hoy inútil en un detector.
5. **No tocar** los tamaños de memoria: nueve días de datos confirman que están bien dimensionados.

2. Estado general del firmware

2.1. Dónde está el proyecto

El firmware del datalogger SPQ sobre STM32L496 está **funcionalmente completo** respecto de su objetivo, que es sustituir al firmware AVR en producción (FWDLGX). Todo el hardware de la placa tiene driver validado en banco, y la capa de aplicación —configuración, protocolo con el servidor, almacenamiento, Modbus, consignas y watchdog— está escrita y validada contra el servidor real.

Bloque	Estado
Arranque, relojes, FreeRTOS, bajo consumo (6 µA en reposo)	Validado
Consola de comandos y diagnóstico	Validado
Memoria EEPROM, reloj de tiempo real, medida de 4-20 mA	Validado
RS-485 / Modbus, contador de pulsos, electroválvula	Validado
microSD (sectores y sistema de archivos)	Validado
Modem LTE y sesión completa con el servidor	Validado
Configuración persistente y contrato de hashes con el servidor	Validado
Almacenamiento de muestras y volcado a microSD	Validado
Cálculo de caudal a partir de pulsos	Funcionando; pendiente de ajuste fino en campo
Doble consigna de presión	Validado
Watchdog	Validado, incluida la cadena completa hasta el servidor
Registro de eventos en microSD	Validado (este informe)

2.2. Lo que se ha probado en corridas largas

Antes de esta instalación el firmware pasó tres pruebas de larga duración en banco:

Prueba	Duración	Resultado
Contador de pulsos contra un generador patrón	48 h	33.085 pulsos emitidos, 33.085 contados: **ni uno perdido**, dormido y despierto
Contador en modo continuo	48 h	Pasó
Registro de eventos en microSD	19 h	Pasó; la recuperación del buffer tras un reinicio cerró al byte
Instalación de campo (este informe)	**188 h**	**Pasó**

2.3. Lo que falta para producción

Nada de lo que falta bloquea una instalación; es trabajo de cierre y de medición.

Pendiente	Naturaleza
Medir el consumo del modem durante una sesión	Es el **82 % del presupuesto de energía** y es el único dato estimado y no medido. Convierte un rango de autonomía de 0,5 a 4,9 años en un número
Corregir el atraso del reloj del kernel	Firmware — ver el plan
Validación sobre binario compilado en modo Release	Procedimiento: hoy todo corre en modo depuración
Decidir el nivel de detección de batería baja (`BOR_LEV`)	Una decisión, no un desarrollo
Período del lazo de control, para ahorrar despertadas	Optimización de consumo de bajo rendimiento esperado
Actualización de firmware desde la microSD	Diseñado y diferido a propósito
Sensor de humedad y detector de agua para cajas IP68	Diseñado; espera el PCB nuevo

2.4. Una advertencia sobre el presupuesto de energía

El trabajo de bajo consumo del microcontrolador **ya rindió todo lo que podía rendir**: en reposo el micro representa el 1,2 % del consumo del equipo y el modem el 82 %. Si el micro consumiera cero, la autonomía pasaría de 2,7 a 2,8 años. Las palancas que quedan son **medir el modem** y **espaciar las sesiones**, no optimizar firmware.

3. La corrida: qué se midió y cómo

3.1. Montaje y configuración

Firmware	FWDLGARM `0.0.19`, compilado el 20/10/2026 10:49
Placa	SPQ_ARM_R1 (la unidad nueva)
Modo de energía	DISCRETO: el modem se enciende sólo para transmitir
Intervalo de muestreo	300 s (5 min)
Intervalo de transmisión	10.800 s (3 h)
Watchdog	**Activo**
Entradas en uso	Contador de pulsos de un caudalímetro, y los dos rieles de alimentación
Registro de eventos	Activado con el archivo `EVENTOS.ON` en la microSD

3.2. El instrumento: el registro de eventos

Es la pieza que hace posible este informe y entró en el firmware el 19 y 20 de octubre, justo antes de la instalación. Funciona así: si al arrancar el equipo encuentra un archivo llamado `EVENTOS.ON` en su microSD, escribe en esa misma tarjeta una bitácora de todo lo que le pasa. Llevarse la tarjeta apaga el registro, de modo que un equipo normal no paga nada por este mecanismo.

Registra, con fecha y hora: cada arranque y su causa, un resumen de salud cada 6 horas, cada sesión con el servidor —cuánto tardó, con qué señal, y si falló, en qué paso—, cada corrección del reloj y cada anomalía del contador de pulsos.

Cada línea lleva dos marcas de tiempo: la fecha del reloj de tiempo real y los segundos transcurridos desde el arranque, que vienen del reloj interno del procesador. Esa redundancia no era el objetivo del diseño, pero resultó ser la herramienta que destapó dos de los tres hallazgos: al comparar las dos columnas se ve cuando un reloj se mueve y el otro no.

3.3. Ventanas de tiempo

Tramo	Desde	Hasta	Duración
Archivo completo	20/10 10:50	29/10 10:09	215,3 h
Instalación y ajustes en sitio	20/10 10:50	20/10 16:21	5,5 h (6 reinicios)
Operación en campo	**20/10 16:21**	**28/10 12:39**	**188,3 h = 7,8 días**
Tras el retiro del equipo	28/10 12:39	29/10 10:09	21,5 h

Los seis reinicios del primer tramo son el día de instalación: ajuste de la configuración y conexión. **A partir de las 16:21 del 20 de octubre el equipo no se reinició una sola vez.**

4. Lo que quedó validado

4.1. Estabilidad: 188 horas sin un reinicio

Es el resultado principal, y pone a prueba el mecanismo más delicado del firmware: el **watchdog**, el circuito que reinicia el equipo si detecta que una tarea se colgó. Su riesgo no es que no actúe: es que reinicie un equipo sano. Un falso positivo en campo se ve como un equipo que no transmite.

En 188 horas con el watchdog activo no hubo un solo falso positivo. Esto era lo único que quedaba por validar del watchdog, porque un banco de pruebas no lo puede demostrar.

4.2. Memoria: invariable

	Resultado
Memoria dinámica libre	**2992 bytes en las 41 lecturas, sin una sola variación**
Pila de la tarea más exigente	se estabilizó en las primeras horas y no se movió más: **28 % de uso**
Resto de las tareas	entre 6 % y 25 % de uso

Esto descarta a la vez una fuga de memoria y un desbordamiento de pila, que son las dos formas en que un sistema embebido falla después de días de funcionamiento. Y confirma la decisión del 12 de octubre de **no agrandar** los tamaños de memoria: los márgenes de hoy son correctos y sirven como instrumento de detección.

4.3. Datos: ni un registro perdido

	Resultado
Registros entregados al servidor	**2.190**
Sesiones con el servidor	**60 exitosas de 63 = 95,2 %**
Acuerdo de configuración con el servidor	61 de 62 sesiones
Registros sobrescritos por falta de espacio	**cero**, en todo el archivo
Pulsos del caudalímetro contados	743.342 (uno cada 0,91 s)

Las tres sesiones fallidas de la ventana de operación son **fallas de red, no del equipo**: dos cortes del enlace a mitad de transmisión y un caso en que el modem no llegó a registrarse en la red. En los tres casos el equipo conservó los datos sin confirmar y los entregó en la sesión siguiente, que es exactamente el comportamiento para el que se diseñó.

4.4. Alimentación: la primera curva real de batería

Nueve días de medición del riel de la batería, tomada por el propio equipo:

	Mínimo diario	Máximo diario
20/10 22:30	12,546 V	12,784 V
28/10 12:26	12,157 V	12,361 V
Variación en 7,6 días	**−389 mV (51 mV/día)**	**−423 mV (56 mV/día)**

La curva es **monótona y suave** en los 41 resúmenes: es una descarga real, no ruido de medición. Es el primer dato de campo sobre el consumo del equipo completo.

Advertencia de lectura: 51 mV/día no se puede extrapolar a una autonomía sin la curva de descarga de esa batería en particular. Las baterías de litio tienen una zona plana amplia y caen abruptamente al final, así que la tensión no es proporcional a la carga restante.

4.5. El riel de 3,3 V: estable

Se mantuvo entre **3,284 y 3,364 V** durante los nueve días, un margen de 80 mV que incluye el ruido de medición. Esto cierra definitivamente el episodio del 9 de octubre, cuando el ajuste de ese riel se había corrido al manipular la placa.

4.6. El retiro del equipo validó, de paso, la degradación sin enlace

Las 21,5 horas posteriores al retiro —con la antena desconectada— ejercitaron un camino que no se había podido probar: **el equipo siguió midiendo y almacenando sin enlace**, sin perder un dato y sin gastar batería inútilmente. Los ocho intentos de sesión abortaron en 19 segundos cada uno, porque el firmware verifica que haya conexión a la red antes de intentar transmitir.

La memoria de muestras pasó de 34 a 243 registros ocupados de 1.984 disponibles. A ese ritmo el equipo **habría aguantado 6 días más** antes de empezar a sobrescribir el dato más viejo.

5. Hallazgos

5.1. PRIORIDAD 1 — El reloj del kernel atrasa 2,3 %, y eso sesga el caudal

5.1.1. El dato

Cada línea del registro lleva la hora del reloj de tiempo real y los segundos contados por el reloj interno del procesador. Comparando las dos columnas en 60 intervalos de 3 horas a lo largo de toda la corrida:

Lo que contó el reloj de tiempo real	185,21 h
Lo que contó el reloj del procesador	180,95 h
Divergencia	**+23.504 ppm = 2,35 %**

Hay que saber cuál de los dos está mal. Lo dice el servidor: cada vez que corrigió la hora del equipo, la corrección fue de **92 segundos cada 60 horas**, o sea que el reloj de tiempo real adelanta 430 ppm. Descontado eso, **el reloj del procesador atrasa 2,31 %**.

5.1.2. Por qué importa

El caudal no se mide contando pulsos: se mide cronometrando el **tiempo entre pulsos**, y ese tiempo lo da el reloj del procesador. Un reloj que atrasa 2,3 % da intervalos más cortos de lo real, y por lo tanto **un caudal 2,3 % más alto de lo real**. Ese número es el que viaja al servidor y el que ve el cliente.

No es ruido: es un **sesgo sistemático** que se acumula en los totalizados del servidor. Y aparece sólo en campo, porque sólo ocurre mientras el equipo duerme.

5.1.3. Por qué es peor de lo que se había medido

El efecto se conocía: se midió en el banco en octubre y dio **1,14 %**. Con ese número se decidió —deliberadamente— salir a campo con el sesgo, porque 1,1 % es menor que la incertidumbre de un caudalímetro.

Esta corrida lo duplica, y la causa es la carga de pulsos. El mecanismo del atraso es que cada vez que el equipo se despierta pierde una fracción de tiempo; cuantas más despertadas, más atraso:

	Pulsos por hora	Atraso del reloj
Prueba de banco, octubre	~300	1,14 %
Esta instalación	**~3.950**	**2,31 %**

Es decir: **el error crece con el caudal que el equipo está midiendo**. Cuanto más trabaja la instalación, más de más reporta. Eso invalida el argumento con el que se había diferido.

5.1.4. La corrección, y una advertencia importante

El arreglo que estaba diseñado desde octubre es **arrastrar el resto de cada despertada a la siguiente**, en lugar de descartarlo. **Ya está hecho** — pero al implementarlo apareció algo que cambia la planificación:

La aritmética no cierra. El reloj del sistema avanza de a 2 unidades por paso, así que lo que se descarta en cada despertada es a lo sumo una unidad: medio milisegundo en promedio. Con las ~4.800 despertadas por hora de esta instalación, eso explica **2,3 segundos por hora de los 83 medidos**:

Lo que explica el resto descartado	**650 ppm**
Lo medido en campo	**23.504 ppm**
Cubierto por este arreglo	**el 3 %**

Falta encontrar el 97 % restante, y por eso este punto quedó a medias a propósito. Hay un sospechoso identificado —cada ciclo de sueño detiene y rearranca el temporizador dos veces, y esa operación no es gratis en este microcontrolador— pero **no se tocó nada sobre esa hipótesis**.

La razón es de método, y el proyecto ya la pagó: en octubre, cuatro “arreglos” sucesivos guiados por un diagnóstico que mentía rompieron código que funcionaba bien. **Primero el instrumento, después el arreglo.**

5.1.5. El instrumento que se agregó, y por qué cierra el caso

El atraso se descubrió cruzando el registro del equipo contra el reloj de tiempo real **y** contra las correcciones del servidor: tres fuentes para medir una cosa, y sólo en una corrida larga.

Ahora lo mide el equipo solo, usando como testigo el **reloj de tiempo real interno del microcontrolador** —un periférico que ya estaba configurado desde el inicio del proyecto y que no se usaba—. Tiene tres propiedades que lo hacen ideal:

- corre del **mismo cristal** que el reloj del sistema, así que la deriva del cristal se cancela y lo que queda es exactamente lo que pierde el mecanismo de sueño;
- **sigue contando mientras el equipo duerme**, que es cuando ocurre la pérdida;
- no cuesta consumo: ya estaba encendido.

El comando de diagnóstico lo informa en dos líneas:

```
atraso tick : +23504 ppm  (RTC interno 12000 s contra tick 11718 s)
              4312 sueños, resto rescatado 2106 cuentas = 2056 ms
```

El número que va a cerrar el caso es el atraso dividido por la cantidad de sueños: los microsegundos que se pierden en cada ciclo apuntan al lugar exacto. Y la misma línea va al registro de eventos cada 6 horas, así que la próxima instalación lo mide sin que nadie la mire.

5.2. PRIORIDAD 2 — El cálculo de deriva del reloj desborda y reporta un número inventado

5.2.1. El dato

```
26/10/26  RELOJ deriva: +11127 s en 30 h  ->  -15791 ppm    (el valor correcto es +103.028)
26/10/26  RELOJ deriva: -11125 s en  3 h  ->  +158022 ppm     (el valor correcto es -
1.030.093)
```

Los dos valores informados son **falsos, y además tienen el signo invertido**: informan que el reloj atrasa cuando adelanta, y al revés.

5.2.2. La causa

Es una línea. El cálculo multiplica el desvío por un millón antes de dividir —correcto, para no perder precisión— pero el resultado **no cabe en el número entero que lo almacena** en cuanto el desvío supera 2.147 segundos. Reproduje la aritmética del microcontrolador y da −16.276 y +162.953, que coinciden con lo informado con un error del 0,1 %: **la causa está confirmada, no es una hipótesis**.

5.2.3. Por qué importa aunque sea un mensaje de diagnóstico

Este proyecto ya pagó tres días de trabajo por un diagnóstico que mentía con autoridad: en octubre, un volcado de registros mal interpretado mandó a buscar un problema que no existía y provocó cuatro "arreglos" sobre código que funcionaba bien.

Un instrumento que informa un número falso es peor que no tener instrumento, porque dirige la búsqueda hacia el lugar equivocado con aparente precisión. Y éste se dispara justo cuando pasa algo grave —un salto de hora grande—, que es cuando más se necesita que diga la verdad.

5.2.4. La corrección

Dos líneas: hacer la multiplicación en un entero de 64 bits. El cálculo correcto ya está escrito.

5.3. PRIORIDAD 3 — El reloj del módulo LTE entregó una hora errada en 3 horas, y se aplicó

5.3.1. El dato

El equipo tiene tres fuentes de hora: su reloj de tiempo real con pila, el reloj que el módulo LTE sincroniza contra la red, y la hora que manda el servidor al confirmar un envío. El 26 de octubre las dos últimas se contradijeron:

```
20:30:47    sesión correcta, el reloj está en hora
23:35:50   el MÓDULO dice que son las 20:30:23  ->  el equipo retrasa el reloj 3 h 5 min
20:30:35    el SERVIDOR dice que son las 23:36:00 ->  el equipo lo vuelve a adelantar
```

El servidor es la referencia correcta, porque es quien archiva las muestras por fecha. Doce segundos después de aceptar la hora del módulo, el equipo la deshizo con la del servidor.

5.3.2. La causa

El módulo entrega la hora con un campo que indica el huso horario. **El firmware no lee ese campo**: da por supuesto que el módulo ya aplicó el huso, porque así se verificó en el banco un día de septiembre. Esta corrida demuestra que no siempre lo hace. Tampoco hay una verificación de la *magnitud* del salto: una diferencia de tres horas se aplica sin objeción.

5.3.3. El riesgo real

Esta vez el daño fue mínimo, por suerte: el servidor corrige en la misma sesión, doce segundos después. Pero el hueco existe. **Si la sesión falla después de aceptar la hora del módulo, el reloj queda tres horas corrido hasta la sesión siguiente que funcione** —y con transmisión cada 3 horas, eso son unas 36 muestras archivadas con fecha equivocada en un servidor que indexa por fecha.

5.3.4. La corrección

Leer el campo del huso y verificar la coherencia del salto antes de aplicarlo: unas quince líneas. El criterio de fondo ya está en el firmware para otros casos —no creerle a un dato que no se puede verificar— y acá falta aplicarlo.

6. Los dos casos que quedan cerrados

6.1. El modem que dejó de transmitir: fue el retiro del equipo

El 28 de octubre a las 12:39 el modem dejó de registrarse en la red y las ocho sesiones siguientes fallaron. **La fecha coincide con el retiro del equipo de la instalación, y la causa es que al retirarlo se desconectó la antena.** No hay nada que investigar en el firmware.

Vale conservar, de todas formas, dos observaciones útiles para el futuro:

La firma de "antena desconectada" quedó documentada. El equipo informó señal **113**, que en la escala del módulo es el valor de "sin señal". Y que ese valor llegara al registro prueba que el módulo **estaba contestando**: lo que faltaba era la red, no el enlace con el módulo ni su alimentación. Es una distinción que no se puede hacer a ojo y que el registro resuelve en una línea.

Y el tramo sin antena validó un camino que no se había probado —la operación sin enlace durante 21 horas— como se detalla más arriba.

6.2. La deriva del reloj de tiempo real: resuelta en el PCB nuevo

El reloj de tiempo real adelantaba **+400 ppm, unos 34 segundos por día**, veinte veces la tolerancia normal de un cristal. La causa está identificada desde septiembre y es de hardware: **el cristal de ese reloj no tiene en esta placa sus condensadores de carga**, y un cristal cargado de menos oscila rápido.

El PCB nuevo los lleva, de modo que el problema se cierra sin tocar el firmware.

Dos consecuencias que sí conviene registrar:

La medición de deriva pasa de ruido a detector. Hoy el firmware avisa en cada corrección que el reloj está fuera de tolerancia, y como está *siempre* fuera, ese aviso no informa nada. Con los condensadores puestos la deriva debería caer a pocas ppm, y entonces ese mismo aviso pasa a ser una alarma con significado: si salta, es que algo cambió.

Y la medición en el PCB nuevo es el criterio de aceptación del cambio de hardware. La deriva la mide el propio equipo contra el servidor, sin instrumentos, así que confirmar que el arreglo funcionó no cuesta nada: basta leer el registro de eventos de la primera semana.

7. Plan de trabajo propuesto

7.1. Prioridades

#	Tarea	Estado	Qué falta
1	Corregir el atraso del reloj del kernel	**Parcial**	Cubierto el 3 %. Falta correr el banco **con el instrumento nuevo** y, con lo que diga, el arreglo de fondo
2	Corregir el desborde del cálculo de deriva	**Hecho**	Validar en banco con un salto grande provocado
3	Validar la hora del módulo antes de aplicarla	**Hecho**	Validar en banco
4	Medir la deriva del reloj en el PCB nuevo	**Preparado**	Esperar el PCB y leer el registro de la primera semana
5	Medir el consumo del modem en una sesión	Pendiente	Es una medición con amperímetro, no firmware

Los puntos 2 y 3 están implementados, compilan sin advertencias en los dos modos de compilación y **se verificaron sin hardware**, reproduciendo la lógica del firmware en el ordenador:

Verificación	Resultado
El cálculo viejo de deriva, con los datos de campo	reproduce **−15791 exacto**, el mismo número que informó el equipo
El cálculo nuevo, con los mismos datos	da el valor correcto
El umbral del desbordamiento	cae en 2147 s, donde se predijo
Los casos que **no** desbordaban	idénticos antes y después: el arreglo no cambia nada donde funcionaba
El lector de la hora del módulo	acepta el caso bueno; rechaza huso cero, otro huso, sin huso y con basura

7.2. Notas sobre cada punto

1. El atraso del reloj es el único hallazgo que afecta un dato que el cliente ve, y por eso se encaró primero aunque sea el más delicado: toca un módulo validado desde el inicio del proyecto.

Quedó **a medias a propósito**: se aplicó la corrección que estaba diseñada —que cubre el 3 % del problema— y se agregó el instrumento que falta para encontrar el resto. El próximo paso es **una corrida de banco con el generador de pulsos a la cadencia de esta instalación**, unos 4.000 pulsos por hora, que es la condición en que el defecto se manifiesta al doble. Con lo que diga el instrumento, el arreglo de fondo sale dirigido en vez de a tientas.

Mientras no esté corregido, hay que tener presente al leer los caudales de los equipos en campo que **vienen entre 1 y 2,5 % altos según la carga de pulsos**, y que una comparación contra el caudalímetro o contra un equipo AVR va a mostrar esa diferencia sin que haya nada roto.

2 y 3 son correcciones acotadas y de bajo riesgo, y las dos mejoran el diagnóstico de campo más que la operación. Pueden ir juntas en una sola entrega.

4 quedó preparado: el cálculo de deriva ya no miente —era la condición para poder creerle— y el mensaje dice ahora qué esperar del PCB nuevo. Es el criterio de aceptación de una modificación del hardware, y sale gratis: el equipo lo mide solo. Conviene dejarlo anotado como algo a revisar en la primera corrida de la placa nueva, porque si los condensadores quedaran mal puestos el síntoma sería el mismo que hoy y nadie lo notaría.

5 es, de lejos, la medición de mayor valor pendiente del proyecto, y no es firmware. El modem es el 82 % del consumo del equipo y es el único componente estimado y no medido: la autonomía calculada varía entre 0,5 y 4,9 años según qué consuma el modem en una sesión. Es **una sola medición** y convierte ese rango en un número.

7.3. Lo que este informe recomienda NO hacer

	Por qué
No agrandar las pilas de las tareas, los buffers ni la memoria dinámica	Nueve días de datos muestran márgenes correctos y estables. Los márgenes actuales funcionan además como detector: si mañana una tarea consume el triple, se ve. Con el doble de margen no se vería nada hasta que fuera tarde
No investigar el modem del 28 de octubre	Fue el retiro del equipo
No tocar el firmware por la deriva del reloj de tiempo real	Es hardware y ya está resuelto en el PCB nuevo
No perseguir los descartes del contador del 22 de octubre	Ver el anexo: fue un episodio acotado, no un problema permanente. Vale observarlo en la próxima instalación

8. Anexos

8.1. Anexo A — Los descartes del contador de pulsos

El firmware descarta los pulsos que llegan demasiado seguidos para ser reales —rebotes del contacto— y los cuenta aparte. En toda la corrida descartó **1.208 pulsos sobre 743.342**, el 0,16 %.

Lo interesante es *cuándo*: no fue un goteo constante, fue **un episodio de siete horas**.

Tramo	Descartes
22/10, de 12:36 a 19:46 — con el pico entre 13:30 y 15:00 (unos 10 por minuto)	1.145
28/10, de 13:07 a 14:03	63
Todo el resto de los nueve días	0

El firmware no registró **ningún** pulso descartado por “demasiado rápido para el caudal máximo”, lo que indica que la configuración del caudalímetro es coherente con la cadencia real.

Un episodio acotado que no se repitió apunta a algo que ocurrió en el sitio, no a un defecto permanente del circuito de entrada. **Queda como observación para la próxima instalación**: si reaparece, hay que mirar el cableado del contacto y el filtro de entrada.

Vale señalar que esta conclusión sólo es posible porque el registro de eventos informa **el incremento** en cada aparición, no sólo el total acumulado. Con el total solo, el resumen diría “1.145 descartes” y nadie sabría que fue una tarde concreta.

8.2. Anexo B — Serie de alimentación, por resumen de 6 horas

Fecha	Horas en marcha	Batería mín..máx (V)	Riel 3,3 V mín..máx (V)
20/10 22:30	6	12,546 .. 12,784	3,304 .. 3,337
21/10 06:14	13	12,513 .. 12,738	3,306 .. 3,337
21/10 18:42	25	12,421 .. 12,665	3,304 .. 3,339
22/10 06:16	37	12,388 .. 12,612	3,301 .. 3,339
22/10 18:40	49	12,342 .. 12,546	3,304 .. 3,341
23/10 06:14	60	12,309 .. 12,540	3,301 .. 3,344
23/10 18:43	72	12,262 .. 12,500	3,301 .. 3,346
24/10 06:14	84	12,256 .. 12,467	3,299 .. 3,341
24/10 18:44	96	12,236 .. 12,447	3,299 .. 3,346
25/10 06:14	107	12,223 .. 12,434	3,301 .. 3,346
25/10 18:42	119	12,203 .. 12,460	3,295 .. 3,344
26/10 06:12	130	12,190 .. 12,408	3,301 .. 3,346
26/10 18:41	143	12,170 .. 12,408	3,293 .. 3,346
27/10 06:18	154	12,170 .. 12,381	3,299 .. 3,346
27/10 18:47	166	12,157 .. 12,381	3,293 .. 3,346
28/10 06:11	177	12,157 .. 12,375	3,299 .. 3,346
28/10 12:26	183	12,157 .. 12,361	3,299 .. 3,346

8.3. Anexo C — Uso de memoria, estabilizado

Tarea	Pila asignada	Mínimo libre observado	Uso máximo
Muestreo (`tkSys`)	1024 palabras	734	28 %
Sesión con el servidor (`tkWan`)	2048 palabras	1779	13 %

Tarea	Pila asignada	Mínimo libre observado	Uso máximo
Consola (`tkCmd`)	1024 palabras	768	25 %
Control (`tkCtl`)	384 palabras	359	7 %
Consigna de presión (`tkCtlPres`)	512 palabras	388	24 %
Memoria dinámica	3000 bytes	**2992 en las 41 lecturas**	0,3 %

Los valores son de un binario compilado en **modo depuración**, que usa más pila que el modo de producción; son por lo tanto el peor caso.

8.4. Anexo D — Sesiones con el servidor

Sesiones exitosas	60
Sesiones fallidas en operación	3 (2 cortes del enlace, 1 sin registro en red)
Tasa de éxito	95,2 %
Duración media de una sesión	56 s (mínimo 13, máximo 97)
Registros por sesión	de 2 a 72
Intentos de conexión necesarios	**1 en las 62 sesiones con enlace**
Señal recibida	media −84 dBm, rango −65 a −93 dBm

El dato de que **nunca hizo falta más de un intento de conexión** indica que la cobertura del sitio, aunque no era buena, era estable.

8.5. Anexo E — Origen de los datos

Todo este informe sale de un único archivo: `EVENTOS.TXT`, 454 líneas y 42.458 bytes, escrito por el propio equipo en su microSD. No se usó ningún instrumento externo.